Python cheatsheet · 19 chapters · only Python code

Selenium with Python: cheatsheet

The key ideas, every annotated line of every example in Python, the tables and the common mistakes from every chapter, in one place for quick revision. Everything here comes from the chapters and shows Python only, so this sheet is all you need. Each card links back to the lesson that teaches it.

92 key ideas | 54 code examples | 10 numbers | 61 tables | 38 tips and traps

STEP 1 Meet Selenium

1.1 What Selenium is and where it came from

What Selenium does, its three parts, the key words, its history to the W3C standard and a first real program.

KEY IDEAS

- **What Selenium is** An umbrella project of tools and libraries that automate real browsers the way a user would. At its core, a browser user agent library.
- **Three parts** WebDriver (library plus browser driver, for code), IDE (record and play back extension), Grid (many sessions on many machines).
- **Not a test framework** Selenium drives the browser. Assertions, test structure and reports come from JUnit, pytest, Mocha and similar tools.
- **Key dates** Selenium 2.0 on 8 July 2011, 3.0 on 13 October 2016, W3C Recommendation on 5 June 2018, Selenium 4.0 on 13 October 2021.
- **Program shape** Start a session, send the browser somewhere, ask or act, and always quit at the end.

CODE

A first look: open a page, read its title, close the browser

`from selenium import webdriver`	Import the webdriver package, which holds one class per browser.
`driver = webdriver.Chrome()`	Start Chrome. Selenium Manager finds a matching chromedriver, then a new session begins.
`driver.get("https://www.selenium.dev/selenium/web/web-form.html")`	Open the page and wait until it has loaded.
`print("Title:", driver.title)`	driver.title asks the browser for the page title.
`print("URL:", driver.current_url)`	current_url asks which address the browser is showing.
`driver.quit()`	End the session, close every window and stop chromedriver.

Lesson: A first look at real code

TABLES

Table: what people use selenium for · an everyday example

What people use Selenium for	An everyday example
Testing web applications	Before a bank releases a new version of its login page, a test opens the page in Chrome, Firefox and Edge, logs in with a test account and checks the balance appears.
Repetitive tasks	A script logs into a supplier portal every morning at 6 and downloads the order report for the day.
Collecting data from pages	A researcher reads prices from a site that has no API. The docs ask you to check the terms of service of the site first, because some sites forbid it and some block Selenium.

Lesson: What Selenium is

Table: part · what it is · use it when

Part	What it is	Use it when
WebDriver	A library for your programming language plus a driver for each browser. Your code sends commands, the driver makes the browser carry them out.	You write automated tests or scripts in Java, Python, JavaScript, C# or Ruby. This is what most of this subject teaches.
IDE	A Chrome and Firefox extension that records what you do in the browser and plays it back as Selenium commands.	You want a quick recording, or you are learning which command matches which action.
Grid	A server that spreads WebDriver sessions over many machines, browsers and operating systems.	One machine is too slow, or you must test on browser and operating system pairs you do not have.

Lesson: WebDriver, IDE and Grid

Table: word · meaning in the selenium documentation

Word	Meaning in the Selenium documentation
API	The set of commands you use to control WebDriver, such as get, findElement and click.
Library	The code module that holds those commands for one language: a .jar file for Java, a pip package for Python, an npm package for JavaScript.
Driver	The program that controls one browser, such as chromedriver for Chrome and geckodriver for Firefox. Browser vendors write most of them. It usually runs on the same machine as the browser.
Framework	An extra library that supports a test suite: test runners such as JUnit, pytest or Mocha, or tools such as Cucumber.

Lesson: WebDriver, IDE and Grid

Table: version · released · what changed

Version	Released	What changed
Selenium 2.0	8 July 2011	The WebDriver APIs became the heart of Selenium. Selenium 1 code still ran.
Selenium 3.0	13 October 2016	The original Selenium Core implementation was removed. Every major browser vendor now ships its own driver.
W3C WebDriver	5 June 2018	WebDriver became a W3C Recommendation, a standard every browser can implement the same way.
Selenium 4.0	13 October 2021	Support for the old JSON Wire Protocol was removed. Only the W3C protocol is used. Relative locators and a rebuilt Grid arrived.

Lesson: How Selenium grew

TIPS AND TRAPS

COMMON MISTAKE Selenium only controls the browser. It does not decide whether a test passed, it does not write reports and it does not give you Given, When, Then syntax. Those come from a test framework such as JUnit, pytest or Mocha. Chapter 5.1 joins the two.

Lesson: What Selenium is

REMEMBER Always end with quit. If a program stops without it, the driver process and the browser keep running in the background and hold memory and ports. On a busy test machine those leftovers pile up quickly.

Lesson: A first look at real code

1.2 Architecture and the W3C WebDriver protocol

How a command travels from your code to the browser over the W3C WebDriver protocol, shown with real HTTP traffic.

KEY IDEAS

- **Three programs** Your code and the binding (local end), the driver (remote end), the browser. The driver is written by the browser vendor.
- **HTTP and JSON** Every command is one HTTP request with a fixed method and path. Answers are {"value": ...} on success or {"value": {"error", "message", "stacktrace"}} on failure.
- **Session id** POST /session returns a session id. Every later path contains it. DELETE /session/{id} ends it.
- **Element reference** Elements travel as {"element-6066-11e4-a52e-4f735466cecf": "id"}. Bindings turn id, name and class name into CSS selectors.
- **BiDi** Classic WebDriver is request and response. WebDriver BiDi adds a WebSocket so the browser can send events. CDP access is temporary until BiDi covers it.

CODE

Drive Chrome with plain HTTP and no Selenium library at all

`import json`	For turning Python dictionaries into JSON text and back.
`import subprocess`	For starting the chromedriver program.
`import urllib.request`	Python's built in HTTP client. No Selenium library is imported anywhere.
`driver = subprocess.Popen(["chromedriver", "--port=9515"], stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)`	Start the driver yourself, the job Selenium usually does for you. Its own messages are hidden.
`time.sleep(1)`	Give it a moment to open port 9515.
`def call(method, path, body=None):`	Send one W3C command and return the part of the answer under "value".
`data = json.dumps(body).encode() if body is not None else None`	POST bodies are JSON objects. GET and DELETE have none.
`reply = json.load(urllib.request.urlopen(req))`	Send it and read the JSON answer.
`print(method, path, " → ", text[:70] + ("..." if len(text) > 70 else ""))`	Print the request and the start of the answer.
`return reply["value"]`	Every success answer keeps its result under "value".
`KEY = "element-6066-11e4-a52e-4f735466cecf"`	The fixed key the W3C spec uses for element references.
`s = "/session/" + call("POST", "/session", {"capabilities": {"alwaysMatch": {"browserName": "chrome"}}})["sessionId"]`	New Session. Chrome opens and we keep the session path.
`call("POST", s + "/timeouts", {"implicit": 2000})`	Set Timeouts: Find Element may keep looking for 2 seconds, so the next page has time to load.
`call("POST", s + "/url", {"url": "https://www.selenium.dev/selenium/web/web-form.html"})`	Navigate To.
`call("GET", s + "/title")`	Get Title.
`box = call("POST", s + "/element", {"using": "css selector", "value": "[name='my-text']"})[KEY]`	Find Element.
`call("POST", s + "/element/" + box + "/value", {"text": "Selenium"})`	Element Send Keys.
`call("POST", s + "/element/" + button + "/click", {})`	Element Click submits the form.
`call("GET", s + "/element/" + message + "/text")`	Get Element Text on the next page.
`call("DELETE", s)`	Delete Session closes Chrome.
`driver.terminate()`	Stop chromedriver.

Lesson: See the wire for yourself

Watch the real requests a normal Selenium script sends

`import logging`	Python's standard logging module.

`logging.basicConfig(level=logging.DEBUG, format="%(message)s", stream=sys.stdout)`	Print DEBUG messages, which include every HTTP request the binding sends.
`for name in ("urllib3", "selenium.webdriver.common"):`	Hide two noisy loggers so only the protocol messages remain.
`driver = webdriver.Chrome()`	POST /session
`driver.get("https://www.selenium.dev/selenium/web/web-form.html")`	POST /session/{id}/url
`box = driver.find_element(By.NAME, "my-text")`	POST /session/{id}/element. Watch how By.NAME becomes a CSS selector.
`box.send_keys("Hi")`	POST /session/{id}/element/{element id}/value
`print("title is", driver.title)`	GET /session/{id}/title
`driver.quit()`	DELETE /session/{id}

Lesson: See the wire for yourself

NUMBERS TO REMEMBER

Success — Exam favourite

HTTP 200 and {"value": result}

The result is null for commands that only act, a string for text and titles, a list for Find Elements.

Lesson: The W3C WebDriver protocol

Error — Exam favourite

HTTP 4xx or 5xx and {"value": {"error", "message", "stacktrace"}}

error is a fixed code such as no such element. The binding turns it into an exception.

Lesson: The W3C WebDriver protocol

Element reference

{"element-6066-11e4-a52e-4f735466cecf": "f.C6…e.6"}

A web element travels as an object with this one fixed key. The value is an opaque id made by the driver.

Lesson: The W3C WebDriver protocol

TABLES

Table: spec word · in practice · example

Spec word	In practice	Example
Local end	The client side, usually a language library on top of the protocol	selenium for Python, selenium-java, selenium-webdriver for npm
Remote end	The server side that carries out the commands	chromedriver, or Selenium Grid
Intermediary node	A remote end that passes commands on to another remote end	Selenium Grid, which forwards to a node
Endpoint node	The last remote end in the chain, built into or next to the browser	chromedriver driving Chrome

Lesson: The pieces between your code and the browser

Table: command · method · path · what you write in python

Command	Method	Path	What you write in Python
New Session	POST	/session	`webdriver.Chrome()`
Delete Session	DELETE	/session/{session id}	`driver.quit()`
Navigate To	POST	/session/{id}/url	`driver.get(url)`
Get Title	GET	/session/{id}/title	`driver.title`
Find Element	POST	/session/{id}/element	`driver.find_element(...)`
Find Elements	POST	/session/{id}/elements	`driver.find_elements(...)`
Element Click	POST	/session/{id}/element/{element id}/click	`element.click()`
Element Send Keys	POST	/session/{id}/element/{element id}/value	`element.send_keys(...)`
Get Element Text	GET	/session/{id}/element/{element id}/text	`element.text`
Set Timeouts	POST	/session/{id}/timeouts	`driver.implicitly_wait(2)`
Execute Script	POST	/session/{id}/execute/sync	`driver.execute_script(...)`
Take Screenshot	GET	/session/{id}/screenshot	`driver.get_screenshot_as_png()`

Lesson: The W3C WebDriver protocol

Table: error code · http status · python exception you see

Error code	HTTP status	Python exception you see
no such element	404	NoSuchElementException
stale element reference	404	StaleElementReferenceException
invalid session id	404	InvalidSessionIdException
element not interactable	400	ElementNotInteractableException
element click intercepted	400	ElementClickInterceptedException
invalid selector	400	InvalidSelectorException
timeout	500	TimeoutException
unexpected alert open	500	UnexpectedAlertPresentException
session not created	500	SessionNotCreatedException

Lesson: The W3C WebDriver protocol

Table: layout · where things run · code difference

Layout	Where things run	Code difference
Direct	Your code, the driver and the browser on one machine.	`new ChromeDriver()`, `webdriver.Chrome()`
Remote	Your code on one machine, driver and browser on another, reached through Selenium Server or RemoteWebDriver.	`new RemoteWebDriver(url, options)`, `webdriver.Remote(url, options=...)`
Grid	Selenium Grid receives the commands and relays them to the driver on a node machine.	Same as remote, the URL points at the Grid

Lesson: Local, remote and Grid, plus BiDi

TIPS AND TRAPS

REMEMBER

The binding changes some locators before they leave your machine. The Python binding sends By.ID "x" as the CSS selector [id="x"], By.NAME as [name="x"] and By.CLASS_NAME as .x. Look for it in the log in the next lesson. That is why the W3C spec only needs five strategies: css selector, link text, partial link text, tag name and xpath.

Lesson: The W3C WebDriver protocol

REMEMBER

You will never write raw HTTP in real tests. The point is to see that nothing magic happens: Selenium is a polite HTTP client with good manners about waiting, errors and types. When a test fails with a strange message, picture the request that was sent and the error code that came back.

Lesson: See the wire for yourself

1.3 Setup, Selenium Manager and your first script

Install Selenium, let Selenium Manager match the driver, and write and tidy up your first real script.

KEY IDEAS

- **Install** Python: pip install selenium (3.10+).
- **Selenium Manager** Ships with Selenium since 4.6. A fallback that discovers the browser version, resolves, downloads and caches the driver in ~/.cache/selenium, and since 4.11 can download browsers.
- **Driver mismatch** An old driver on PATH or given by hand gives session not created. Delete it and let Selenium Manager match the driver.
- **Eight components** Start session, act on browser, ask the browser, wait strategy, find element, act on element, ask the element, end session.
- **Always quit** Put quit in finally, or use a Python with block. quit closes every window and stops the driver, close only closes one window.

CODE

What an old driver on a new browser looks like

`from selenium.webdriver.chrome.service import Service`	Service describes how to start the driver program.
`service = Service(executable_path="/opt/drivers/chromedriver-147")`	Point Selenium at an old chromedriver 147 on purpose. Selenium Manager is now skipped.
`driver = webdriver.Chrome(service=service)`	Chrome on this machine is version 153, so New Session fails.
`except SessionNotCreatedException as e:`	The driver answered with the W3C error code session not created.
`print(e.msg.split("\n")[0])`	Print the first line of the driver's message.

Lesson: Selenium Manager finds the driver

The eight basic components of a Selenium script

`from selenium.webdriver.common.by import By`	By holds the locator strategies: By.NAME, By.CSS_SELECTOR and more.
`driver = webdriver.Chrome()`	1. Start the session. Chrome opens with a fresh profile.
`driver.get("https://www.selenium.dev/selenium/web/web-form.html")`	2. Take action on the browser: go to the practice form.
`title = driver.title`	3. Request browser information: the page title.
`driver.implicitly_wait(0.5)`	4. Establish a waiting strategy. Python takes seconds, so 0.5 means half a second.
`text_box = driver.find_element(by=By.NAME, value="my-text")`	5. Find an element by its name attribute.
`submit_button = driver.find_element(by=By.CSS_SELECTOR, value="button")`	5. Find the first button with a CSS selector.
`text_box.send_keys("Selenium")`	6. Take action on an element: type into the box.
`submit_button.click()`	6. Click Submit. The browser loads the next page.
`message = driver.find_element(by=By.ID, value="message")`	Find the message on the new page.
`value = message.text`	7. Request element information: its visible text.
`driver.quit()`	8. End the session. The browser and the driver both close.

Lesson: The eight steps of a first script

Quit even when something fails

`with webdriver.Chrome() as driver:`	A with block calls driver.quit() for you when the block ends, even after an error.
`driver.find_element(By.ID, "no-such-id")`	This element does not exist, so this line raises.
`except NoSuchElementException as e:`	Catch the error so we can report it.
`print("Failed:", e.msg.split("\n")[0])`	e.msg is the message from the driver. Keep its first line.
`print("Browser closed")`	We are outside the with block, so quit has already run.

Lesson: Always end the session

TABLES

Table: language · minimum version for selenium 4.51 · how you add selenium

Language	Minimum version for Selenium 4.51	How you add Selenium
Python	Python 3.10	`pip install selenium`, or a line `selenium==4.51.0` in requirements.txt

Lesson: Install Selenium for your language

Table: setting · default · what it does

Setting	Default	What it does
`cache-path`	`~/.cache/selenium`	Where drivers, browsers and metadata are stored
`ttl`	3600 seconds	How long resolved versions are trusted before asking the vendor again
`offline`	false	No network requests or downloads at all
`avoid-stats`	false	Selenium Manager sends anonymous usage numbers to Plausible once a day. Set SE_AVOID_STATS=true to stop it.
`proxy`	none	An HTTP proxy for the downloads

Lesson: Selenium Manager finds the driver

Table: step · what it means · in this example

Step	What it means	In this example
1. Start the session	Create a driver. A browser opens.	Chrome
2. Take action on the browser	Navigate, go back, resize and so on.	Open the web form
3. Request browser information	Title, address, window handles, cookies.	Read the title
4. Establish a waiting strategy	Make sure an element is present before you look for it.	Implicit wait of 500 ms
5. Find an element	Most commands are about elements.	The text box and the button
6. Take action on an element	Type, click, clear.	Type Selenium, click Submit
7. Request element information	Text, attributes, size, state.	Read the message
8. End the session	Quit the driver and close the browser.	quit

Lesson: The eight steps of a first script

Table: quit · close

	quit	close
Closes	Every window and tab of the session	Only the current window or tab
Driver process	Stopped	Keeps running while other windows are open
After it	The driver object cannot be used again	Switch to another window to carry on

Lesson: Always end the session

TIPS AND TRAPS

REMEMBER Use a virtual environment in Python and a package.json in JavaScript. Each project then pins its own Selenium version, so upgrading one project never breaks another.

Lesson: Install Selenium for your language

COMMON MISTAKE If you pass a driver path yourself, Selenium trusts you and never calls Selenium Manager. The documentation says that with Selenium 4.6 or newer you should not need to set a driver location at all. Delete old drivers from PATH and let Selenium Manager do the job.

Lesson: Selenium Manager finds the driver

COMMON MISTAKE The documentation itself warns that an implicit wait is rarely the best solution. It is used here because it is the easiest to show. Chapter 4.1 replaces it with explicit waits.

Lesson: The eight steps of a first script

REMEMBER In a test framework the setup and teardown hooks do this job: start the driver before each test and quit it after each test, pass or fail. Chapter 5.1 shows the hooks in JUnit 5, pytest and Mocha.

Lesson: Always end the session

STEP 2 Control the browser

2.1 Sessions, options and timeouts

Options and capabilities, page load strategy, the three session timeouts and the driver service, shown with real runs.

KEY IDEAS

- **Options** Since Selenium 4, describe sessions with Options classes. They become W3C capabilities, and browser specific ones carry a prefix such as goog:chromeOptions.
- **Page load strategy** normal waits for complete (load), eager for interactive (DOMContentLoaded), none for the first download. It covers navigation by address, not clicks.
- **Default timeouts** Implicit 0, page load 300,000 ms, script 30,000 ms.
- **Service** Starts and stops a local driver: file, port, arguments and logs. Not used for remote sessions.
- **Granted capabilities** Read them after New Session to learn the real browser version, platform and defaults such as dismiss and notify.

CODE

Ask for a session with options, then read what the browser agreed to

`options = webdriver.ChromeOptions()`	An Options object describes the session you want. browserName is already chrome.
`options.add_argument("--headless=new")`	A Chrome command line switch: run without a visible window.
`options.add_argument("--window-size=1280,800")`	Another switch: the size of the browser window.
`options.page_load_strategy = "eager"`	A W3C capability: get() returns at DOMContentLoaded.
`driver = webdriver.Chrome(options=options)`	New Session is sent with these capabilities.
`caps = driver.capabilities`	A plain dict of what the driver actually granted.
`for key in ("browserName", "browserVersion", "platformName", "pageLoadStrategy", "unhandledPromptBehavior"):`	Print a few of the standard keys.

Lesson: Options and capabilities

Read the three session timeouts, then trip one on purpose

`t = driver.timeouts`	The timeouts of this session. Python reports them in seconds.

`print("implicit:", t.implicit_wait)`	0: find_element does not wait at all unless you ask.
`print("pageLoad:", t.page_load)`	300 seconds, which is 300,000 ms.
`print("script:", t.script)`	30 seconds.
`driver.set_script_timeout(1)`	Allow scripts only one second from now on.
`driver.execute_async_script("/* never calls its callback */")`	An async script must call its callback. This one never does.
`except TimeoutException as e:`	Python raises TimeoutException for the W3C error script timeout.

Lesson: The three session timeouts

Start the driver on a port you choose and ask if it is ready

`service = Service(port=9600)`	A Service describes how to start chromedriver: here, on port 9600.
`driver = webdriver.Chrome(service=service)`	Selenium starts chromedriver on that port, then opens a session.
`print("Driver URL:", service.service_url)`	The address the binding sends its HTTP commands to.
`with urllib.request.urlopen("http://localhost:9600/status") as reply:`	GET /status is a W3C command that needs no session.
`print("ready:", status["ready"])`	The driver says whether it can start new sessions.
`driver.quit()`	Ends the session and stops the service.

Lesson: The driver service

NUMBERS TO REMEMBER

Implicit wait

0

How long Find Element keeps looking before it gives up with no such element. Zero means it checks once.

Lesson: The three session timeouts

Page load Exam favourite

300,000 ms

Five minutes. How long a navigation may take before a timeout error.

Lesson: The three session timeouts

Script

30,000 ms

Thirty seconds. How long an injected script may run before the script timeout error.

Lesson: The three session timeouts

TABLES

Table: capability · what it controls · default

Capability	What it controls	Default
`browserName`	Which browser. Set for you by each Options class.	set by the class
`browserVersion`	Which version. Selenium Manager can download it if missing.	whatever is installed
`platformName`	The operating system at the remote end. Matters most on Grid and in the cloud.	the local OS
`pageLoadStrategy`	When navigation commands return: normal, eager or none.	normal
`acceptInsecureCerts`	Trust expired or self signed TLS certificates.	false
`timeouts`	Script, page load and implicit wait.	30 s, 300 s, 0
`unhandledPromptBehavior`	What happens to an alert your code did not handle.	dismiss and notify
`proxy`	Send browser traffic through a proxy server.	none
`setWindowRect`	Whether the browser supports resizing and moving windows.	reported by the driver
`strictFileInteractability`	Strict checks for hidden file inputs.	off

Lesson: Options and capabilities

Table: chrome switch · what it does

Chrome switch	What it does
`--headless=new`	No visible window. Common on build servers.
`--start-maximized`	Open the window maximised.
`--window-size=1280,800`	A fixed window size, so pages lay out the same on every machine.
`--user-data-dir=...`	Use a chosen profile folder instead of a fresh temporary one.

Lesson: Options and capabilities

Table: strategy · waits until readystate is · browser event · use it when

Strategy	Waits until readyState is	Browser event	Use it when
normal (default)	complete	load: every image, font and style sheet is in	You want the safest behaviour
eager	interactive	DOMContentLoaded: the HTML is parsed, images may still load	Large images or ads slow the page and the test does not need them
none	any	only the first download of the page	You control every wait yourself

Lesson: Page load strategy

Table: language · set the page load timeout to 30 s

Language	Set the page load timeout to 30 s
Python	`driver.set_page_load_timeout(30)`

Lesson: The three session timeouts

Table: service setting · why you would change it

Service setting	Why you would change it
Driver location	Only with Selenium older than 4.6 or a special setup. Otherwise let Selenium Manager find it.
Port	A firewall allows only certain ports, or you want to talk to the driver yourself.
Log output and level	Driver logs are thrown away unless you send them somewhere. Levels are ALL, DEBUG, INFO, WARNING, SEVERE and OFF.
Arguments	Pass chromedriver switches, such as appending to a log file.

Lesson: The driver service

TIPS AND TRAPS

REMEMBER Each Options class can also take the binary path of a Chromium based browser, extensions, and excludeSwitches to turn off a default switch of chromedriver. The documentation lists them on the Chrome page. Use them only when you need them, because every extra setting is one more difference from what real users run.

Lesson: Options and capabilities

COMMON MISTAKE Switching to eager or none to make tests faster is a trade. Every element that arrives later now needs an explicit wait, or the test becomes flaky. Chapter 4.1 shows how to write those waits.

Lesson: Page load strategy

IN THE EXAM These timeouts belong to the browser session. Explicit waits such as WebDriverWait are different: they run in your code and poll the browser. Mixing an implicit wait with explicit waits can give surprising wait times, which chapter 4.1 explains.

Lesson: The three session timeouts

2.2 Navigation, window size, screenshots and PDF

Navigation and history, title and URL, window size and position, screenshots and printing pages to PDF.

KEY IDEAS

- **Navigation** get or navigate().to, back, forward, refresh. Python spells them driver.back(), driver.forward(), driver.refresh().
- **Where am I** Title and current URL. Check them when a test depends on a redirect or on history, because back on the first page raises no error.
- **Window rect** Set and get size and position. An exact size is more portable than maximize, which needs a window manager.
- **Screenshots** Take Screenshot (viewport) and Take Element Screenshot return base64 PNG. Save one when a test fails, before quit.
- **Print Page** Returns a base64 PDF. Use PrintOptions for orientation, ranges, size, margins and scale. Chromium needs headless.

CODE

Go somewhere, go back, go forward, reload

`driver.get("https://www.selenium.dev/selenium/web/web-form.html")`	Navigate To. Waits for the page load strategy.
`driver.get("https://www.selenium.dev/selenium/web/xhtmlTest.html")`	A second page. History now holds two entries.
`print("2", driver.title, "at", driver.current_url)`	Title and address of the page now showing.
`driver.back()`	Press the Back button.
`driver.forward()`	Press the Forward button.

`driver.refresh()`	Reload the current page.

Lesson: Navigate and ask where you are

Set the window size and position, then read them back

`driver.set_window_size(1024, 768)`	Restore the window and make it 1024 by 768 pixels.
`driver.set_window_position(40, 30)`	Move its top left corner to x 40, y 30 on the screen.
`size = driver.get_window_size()`	A dict with width and height.
`where = driver.get_window_position()`	A dict with x and y.
`print("rect", driver.get_window_rect())`	One W3C command, Get Window Rect, returns all four numbers at once.

Lesson: Window size and position

Save a screenshot of the page and of one element

`def png_size(path):`	A PNG keeps its width and height at bytes 16 to 24 of the file.
`driver.save_screenshot("page.png")`	Take Screenshot of the viewport and save it as a PNG.
`button.screenshot("button.png")`	Take Element Screenshot: just the area of the button.

Lesson: Screenshots

Print the page to a PDF file

`options.add_argument("--headless=new")`	Chromium prints to PDF only in headless mode.
`print_options = PrintOptions()`	Paper settings: orientation, page ranges, size, margins, scale, background.
`pdf = base64.b64decode(driver.print_page(print_options))`	The W3C Print Page command answers with base64, so decode it.
`print("page.pdf starts with", pdf[:5].decode())`	Every PDF file starts with %PDF-.

Lesson: Print a page to PDF

TABLES

Table: what you want · python

What you want	Python
Open an address	`driver.get(url)`
Back	`driver.back()`
Forward	`driver.forward()`
Reload	`driver.refresh()`
Page title	`driver.title`
Current address	`driver.current_url`

Lesson: Navigate and ask where you are

Table: command · what it does

Command	What it does
Set window size	Restores the window if it was maximised, then sets width and height in pixels
Set window position	Moves the top left corner to x and y on the screen
Get window rect	Width, height, x and y in one answer
Maximize	Makes the window fill the screen, without covering the system menus
Minimize	Hides the window, usually to the taskbar. The exact result depends on the window manager. Selenium 4 and later.
Fullscreen	Fills the whole screen, like pressing F11

Lesson: Window size and position

Table: language · page screenshot to file · element screenshot to file

Language	Page screenshot to file	Element screenshot to file
Python	`driver.save_screenshot("page.png")`	`element.screenshot("el.png")`

Lesson: Screenshots

Table: printoptions setting · example

PrintOptions setting	Example
Orientation	portrait or landscape
Page ranges	"2-4"
Page size	A4, Legal and others
Margins	top, bottom, left, right
Scale	1.0 is 100 %, 0.25 is 25 %
Background	whether background colours and images print
Shrink to fit	whether wide content is shrunk to the page

Lesson: Print a page to PDF

TIPS AND TRAPS

COMMON MISTAKE After refresh or any navigation, the elements you found before belong to the old page. Using them raises stale element reference. Find them again on the new page. Chapter 5.4 covers this error in depth.

Lesson: Navigate and ask where you are

COMMON MISTAKE Maximize and minimize need a desktop window manager. On a bare Linux build server they can fail with an unknown error, which is what happened when this page was built. Setting an exact size with Set Window Rect or the --window-size switch works everywhere and gives the same layout on every machine.

Lesson: Window size and position

REMEMBER Take a screenshot in the teardown of a failing test, before quit. Name the file after the test, so the report can link to it. Chapter 5.1 wires this into JUnit, pytest and Mocha.

Lesson: Screenshots

STEP 3 Find and use elements

3.1 Locators: how Selenium finds an element

All eight locator strategies, CSS and XPath, relative locators and how to choose a locator that lasts.

KEY IDEAS

- **Eight strategies** id, name, class name, css selector, xpath, tag name, link text, partial link text. find element returns the first match.
- **Five on the wire** W3C defines css selector, link text, partial link text, tag name and xpath. Bindings turn id, name and class name into CSS.
- **CSS and XPath** CSS: #id, .class, tag[attr="v"], parent > child. XPath adds text matching and moving up to a parent. Prefer relative XPath (//).
- **Relative locators** above, below, toLeftOf, toRightOf and near (50 px), built on getBoundingClientRect. They depend on layout, so check what they return.
- **Preference** Unique stable id, then a short CSS selector, then XPath. Avoid absolute paths, styling classes and generated ids. Consider data-test attributes.

CODE

All eight locator strategies on the documentation example page

`def show(how, e):`	A helper that prints which element a locator found.
`show("CLASS_NAME", driver.find_element(By.CLASS_NAME, "information"))`	The first element whose class list contains information.
`show("CSS_SELECTOR", driver.find_element(By.CSS_SELECTOR, "#fname"))`	CSS: # means id, so the input with id fname.
`show("ID", driver.find_element(By.ID, "lname"))`	The element whose id attribute is lname.
`show("NAME", driver.find_element(By.NAME, "newsletter"))`	The element whose name attribute is newsletter.
`show("LINK_TEXT", driver.find_element(By.LINK_TEXT, "Selenium Official Page"))`	A link whose whole visible text matches.
`show("PARTIAL_LINK_TEXT", driver.find_element(By.PARTIAL_LINK_TEXT, "Official"))`	A link whose visible text contains Official.
`show("TAG_NAME", driver.find_element(By.TAG_NAME, "a"))`	The first a element on the page.
`show("XPATH", driver.find_element(By.XPATH, "//input[@value='f']"))`	XPath: any input whose value attribute is f.

Lesson: The eight locator strategies

Relative locators: find an element by where it sits

`from selenium.webdriver.support.relative_locator import locate_with`	locate_with() starts a relative locator.
`print(how, "→", e.get_attribute("outerHTML"))`	outerHTML shows the whole tag that was found.
`show("below last name label", driver.find_element(locate_with(By.TAG_NAME, "input").below({By.CSS_SELECTOR: "label[for='lname']"})))`	Python writes the anchor locator as a small dict.
`show("above lname", driver.find_element(locate_with(By.TAG_NAME, "input").above({By.ID: "lname"})))`	An input above the lname box.
`show("right of Male", driver.find_element(locate_with(By.TAG_NAME, "input").to_right_of({By.CSS_SELECTOR: "input[value='m']"})))`	An input to the right of the Male radio.

`show("below fname and above the checkbox", driver.find_element(locate_with(By.TAG_NAME, "input").below({By.ID: "fname"}).above({By.NAME: "newsletter"})))`	Chained: both conditions must hold.
`show("near Newsletter label", driver.find_element(locate_with(By.TAG_NAME, "input").near({By.CSS_SELECTOR: "label[for='newsletter']"})))`	near means at most 50 pixels away. Look at what it really returned.
`show("left of password", driver.find_element(locate_with(By.TAG_NAME, "input").to_left_of({By.ID: "password-field"})))`	On the login page the three inputs sit in one row.

Lesson: Relative locators

TABLES

Table: strategy · finds · example

Strategy	Finds	Example
id	The element whose id attribute matches	`lname`
name	Elements whose name attribute matches	`newsletter`
class name	Elements whose class list contains the value. Compound names such as "a b" are not allowed.	`information`
css selector	Elements matching a CSS selector	`#fname`
xpath	Elements matching an XPath expression	`//input[@value='f']`
tag name	Elements with that tag	`a`
link text	Links whose visible text is exactly the value	`Selenium Official Page`
partial link text	Links whose visible text contains the value	`Official`

Lesson: The eight locator strategies

Table: to find · css selector · xpath

To find	CSS selector	XPath
By id	`#fname`	`//*[@id="fname"]`
By class	`.information`	`//*[contains(@class,"information")]`
Tag with attribute	`input[name="fname"]`	`//input[@name="fname"]`
Attribute starts with	`a[href^="https"]`	`//a[starts-with(@href,"https")]`
Attribute contains	`a[href*="selenium"]`	`//a[contains(@href,"selenium")]`
Direct child	`form > input`	`//form/input`
Any descendant	`form input`	`//form//input`
Second matching input	`input:nth-of-type(2)`	`(//input)[2]`
By visible text	not possible	`//label[text()="Last name:"]`
The parent	not possible	`//input[@id="lname"]/..`
Next sibling	`label + input`	`//label[@for="lname"]/following-sibling::input[1]`

Lesson: CSS selectors and XPath

Table: method · finds an element that is · note

Method	Finds an element that is	Note
above	higher on the page than the anchor	
below	lower on the page than the anchor	
toLeftOf / to_left_of	left of the anchor	
toRightOf / to_right_of	right of the anchor	
near	at most 50 pixels from the anchor	Useful for an input next to its label

Lesson: Relative locators

Table: fragile locator · why it breaks · better

Fragile locator	Why it breaks	Better
`/html/body/div[2]/form/input[3]`	Any new div or input shifts the path	`#lname`
`.btn.btn-primary.mt-3`	Styling classes change when the design changes	`[data-test="submit"]`
`//button[text()="Submit"]`	Breaks when the text is translated or reworded	An id or a data attribute
`#ember1234`	Generated ids change on every page load	A stable attribute near it

Lesson: Choosing a good locator

TIPS AND TRAPS

REMEMBER find element (singular) returns the first match or raises no such element. find elements (plural) returns every match in page order, or an empty list. Chapter 3.2 covers the difference in detail.

Lesson: The eight locator strategies

COMMON MISTAKE Each relative locator sends the page a script to run, so it is slower than a plain CSS selector. Prefer an id or a short CSS selector, and keep relative locators for elements with no good attributes.

Lesson: Relative locators

IN THE EXAM Keep locators in one place, apart from the steps that use them. The documentation recommends declaring them separately, which is exactly what a page object does. Chapter 5.2 shows the pattern.

Lesson: Choosing a good locator

3.2 Finding elements: one, many, scoped and in shadow DOM

Singular and plural finds, scoped searches and the XPath trap, shadow DOM and the active element.

KEY IDEAS

- **One or many** find element returns the first match or raises no such element. find elements returns all matches or an empty list.
- **Search context** You can search from the driver, from an element or from a shadow root. CSS from an element stays inside it.
- **XPath trap** An XPath starting with // always starts at the document, even when called on an element. Use .// to search below the element.
- **Shadow DOM** Find the host, get its shadow root, then search with a CSS selector. Chrome refuses XPath there.
- **Active element** switchTo().activeElement() returns the focused element. Use it to test keyboard flow such as Tab order.

CODE

One element, all elements, and what happens when nothing matches

```
inputs = driver.find_elements(By.TAG_NAME, "input")
```

Plural: a list of every input, in page order.

`for e in inputs:`	Walk the list like any Python list.
`none = driver.find_elements(By.ID, "email")`	No match: an empty list, never an error.
`driver.find_element(By.ID, "email")`	Singular with no match: an exception.

Lesson: Find one or find many

Search inside one element, and the XPath trap

`form = driver.find_element(By.TAG_NAME, "form")`	First find the parent element.
`print("inputs in form:", len(form.find_elements(By.CSS_SELECTOR, "input")))`	Called on the form, CSS only looks inside it.
`print("links via //a:", len(form.find_elements(By.XPATH, "//a")))`	The trap: // starts at the top of the document, so the link outside the form is found.
`print("links via .//a:", len(form.find_elements(By.XPATH, ".//a")))`	The dot means start here. Now nothing outside the form counts.
`last = form.find_element(By.ID, "lname")`	Two commands: find the form, then find inside it.
`same = driver.find_element(By.CSS_SELECTOR, "form #lname")`	The optimised version: one CSS selector, one command.
`print("same element:", last == same)`	Both references point at the same element.

Lesson: Search inside an element

Reach into a shadow root

`print("from the page:", len(driver.find_elements(By.CSS_SELECTOR, box)))`	The checkbox lives in a shadow tree, so a normal search finds nothing.
`host = driver.find_element(By.CSS_SELECTOR, "custom-checkbox-element")`	The shadow host: the element the hidden tree is attached to.
`shadow_root = host.shadow_root`	Get Element Shadow Root. A ShadowRoot object with find_element and find_elements.
`checkbox = shadow_root.find_element(By.CSS_SELECTOR, box)`	Search inside the shadow tree.
`shadow_root.find_element(By.XPATH, ".//input")`	XPath inside a shadow root.
`print("xpath in shadow root:", e.msg.split("\n")[0])`	Chrome refuses it.

Lesson: The shadow DOM

Which element has the focus?

`print("at start:", driver.switch_to.active_element.tag_name)`	Nothing is focused yet, so the active element is the body.
`driver.find_element(By.NAME, "my-text").click()`	Clicking a text box gives it the focus.
`driver.switch_to.active_element.send_keys(Keys.TAB)`	Press Tab on whatever has the focus, as a user would.
`print("after Tab:", driver.switch_to.active_element.get_attribute("name"))`	The focus moved to the next field.

Lesson: The active element

TABLES

Table: find element (singular) · find elements (plural)

	find element (singular)	find elements (plural)
Returns	One element: the first match in page order	A list of every match, in page order
No match	Raises no such element	Returns an empty list
W3C command	Find Element	Find Elements
Use it for	An element that must be there	Counting, looping, or checking that something is absent

Lesson: Find one or find many

TIPS AND TRAPS

COMMON MISTAKE find elements never waits for the first element to appear unless an implicit wait is set, and then it returns as soon as at least one matches. An empty list can mean the page is not ready yet. Chapter 4.1 shows how to wait properly.

Lesson: Find one or find many

REMEMBER Searching inside a container is still very useful when the page repeats a pattern: the Delete button inside the third row of a table, or the price inside one product card. Find the row or card first, then the button inside it.

Lesson: Search inside an element

REMEMBER A shadow root can contain another shadow host. Then you repeat the two steps: get that host, get its shadow root, search again. Each level is one more pair of commands.

Lesson: The shadow DOM

3.3 Interacting with elements and reading them

Click, type, clear, the checks before every action and their errors, element information, select lists and uploads.

KEY IDEAS

- **Five commands** click, send keys, clear, submit and select. Prefer clicking the submit button over submit(). send keys appends, so clear first.
- **Built in checks** Before acting the driver scrolls the element into view and checks it is interactable. Errors: element not interactable, invalid element state, element click intercepted.
- **Attribute vs property** DOM attribute is the HTML, DOM property is the live value. The classic getAttribute mixes both.
- **Select and upload** Select works only on select elements: by visible text, value or index. Upload by sending the full path to input type="file".

CODE

Fill in a form: clear, type, tick and click

`driver.implicitly_wait(2)`	As in chapter 1.3: let finds retry for up to 2 seconds, so the page after Submit has time to load.
`text.send_keys("first try")`	Element Send Keys: types into the box, adding to what is already there.
`text.clear()`	Element Clear: empties the box.
`text.send_keys("Selenium")`	Type the real value.
`print("check 2 before:", check2.is_selected())`	is_selected reads whether a checkbox is ticked.
`check2.click()`	Element Click: ticks it, as a mouse click would.
`driver.find_element(By.ID, "my-radio-2").click()`	Clicking the second radio button unselects the first one.
`driver.find_element(By.CSS_SELECTOR, "button").click()`	Click Submit. The documentation prefers this to the submit() method.

Lesson: Click, type and clear

The three errors interactions give, made on purpose

`def attempt(what, step):`	Run one step and report the error it raises, if any.
`except WebDriverException as e:`	Every Selenium error is a WebDriverException.
`attempt("type into disabled", lambda: driver.find_element(By.NAME, "my-disabled").send_keys("x"))`	A disabled input cannot take keyboard input.
`attempt("clear readonly", lambda: driver.find_element(By.NAME, "my-readonly").clear())`	A readonly input cannot be reset.

`driver.execute_script(`	Add a cookie banner that covers the bottom of the page, as many real sites do.
`attempt("click under banner", lambda: driver.find_element(By.CSS_SELECTOR, "button").click())`	The banner sits over the centre of the button.

Lesson: Click, type and clear

Ask an element about itself

`text.send_keys("hello")`	Type, so the live value differs from the HTML.
`print("disabled box enabled:", driver.find_element(By.NAME, "my-disabled").is_enabled())`	The disabled input reports False.
`print("tag:", text.tag_name, ", rect:", text.rect["width"], "x", text.rect["height"])`	Get Element Rect: position and size in pixels.
`print("get_attribute(value):", text.get_attribute("value"))`	The classic method: property first, then attribute.
`print("get_dom_attribute(value):", text.get_dom_attribute("value"))`	Only what the HTML says. The HTML has no value attribute.
`print("get_property(value):", text.get_property("value"))`	The live JavaScript property: what is typed now.
`print("get_dom_attribute(myprop):", text.get_dom_attribute("myprop"))`	A made up attribute in the HTML.
`print("button colour:", driver.find_element(By.CSS_SELECTOR, "button").value_of_css_property("color"))`	The computed CSS value.
`print("heading text:", driver.find_element(By.TAG_NAME, "h1").text)`	The rendered, visible text.

Lesson: Read information about an element

Work with single and multiple select lists

`from selenium.webdriver.support.select import Select`	Select lives in the support package.
`single = Select(driver.find_element(By.NAME, "selectomatic"))`	Wrap a select element in a Select object.
`print("options:", len(single.options))`	options lists every option element.
`single.select_by_visible_text("Four")`	Pick by the text the user sees.
`single.select_by_value("two")`	Pick by the value attribute.
`single.select_by_index(0)`	Pick by position, counting from 0.
`multi = Select(driver.find_element(By.ID, "multi"))`	A select with the multiple attribute.
`print("multi starts with:", len(multi.all_selected_options), "selected")`	Eggs and Sausages are selected in the HTML.
`multi.deselect_by_value("eggs")`	Only multiple lists can deselect.

Lesson: Select lists

Upload a file without the file dialog

`file.write_text("quarterly numbers")`	Make a small file to upload.
`upload = driver.find_element(By.NAME, "my-file")`	The input with type="file".
`upload.send_keys(str(file.resolve()))`	Send the full path. No dialog opens, the file is simply attached.
`print("browser shows:", upload.get_property("value"))`	Browsers hide the real folder and show a made up folder called fakepath.

Lesson: File upload

TABLES

Table: command · works on · watch out for

Command	Works on	Watch out for
click	Any element	It clicks the centre of the element. If something else is on top there, you get element click intercepted.
send keys	Text fields and other keyboard interactable elements	It adds to the existing text. Clear first if you want to replace it.
clear	Editable, resettable elements: text inputs and content editable	Readonly and disabled inputs give invalid element state.
submit	Form elements	In Selenium 4 it runs a script, so the documentation recommends clicking the submit button instead.
select	select and option elements	Use the Select class, covered below.

Lesson: Click, type and clear

Table: question · python

Question	Python
Is it displayed?	`is_displayed()`
Is it enabled?	`is_enabled()`
Is it selected?	`is_selected()`
Tag name	`tag_name`
Position and size	`rect`
Computed CSS	`value_of_css_property(name)`
Visible text	`text`

Lesson: Read information about an element

Table: method · reads · use it for

Method	Reads	Use it for
get DOM attribute	Only the attribute in the HTML	Checking what the server sent, such as a data-test value
get DOM property	Only the live JavaScript property	What is in a field now, whether a box is checked now
get attribute (classic)	The property if there is one, otherwise the attribute. Selenium does this with a JavaScript atom	Old code. Prefer the two precise methods.

Lesson: Read information about an element

Table: rule from the documentation · what it means for you

Rule from the documentation	What it means for you
Select only works on select and option elements	Drop downs built from div or li with JavaScript need normal clicks instead
Since Selenium 4.5 a Select cannot be created on a disabled select	Check isEnabled first if the list may be disabled
Disabled options cannot be selected	Trying raises an error
Only multiple lists can deselect	deselect on a single list raises an error

Lesson: Select lists

TIPS AND TRAPS

REMEMBER When the test runs on a Grid, the file must reach the remote machine. The Java, Python and JavaScript bindings handle this with a file detector that uploads the file to the node first. Chapter 6.1 shows it.

Lesson: File upload

STEP 4 Synchronise and control

4.1 Waiting strategies: implicit, explicit and fluent

Why tests race the page, implicit and explicit waits, expected conditions, custom waits and why not to mix them.

KEY IDEAS

- **Implicit wait** A session setting, default 0. Every find keeps looking up to that long and returns as soon as the element appears. It only waits for presence.
- **Explicit wait** A loop with a condition, timeout and polling interval.
- **Expected conditions** presence, visibility, clickable, text present, title contains, staleness. Wait for visibility before typing.
- **Never mix** Implicit plus explicit makes waits unpredictable: 10 s with 15 s can time out after about 20 s. Keep implicit at 0 and use explicit waits.

CODE

The race: look for the box straight after the click

`driver.find_element(By.ID, "adder").click()`	The page adds a red box, but only 1 second after the click.
`driver.find_element(By.ID, "box0")`	Our code asks at once. No wait is set, so the driver checks only once.
`print("too early: no such element")`	Selenium was faster than the page.

Lesson: The race between your code and the page

Implicit wait: every find keeps looking

`driver.implicitly_wait(2)`	For the whole session: a find may keep looking for up to 2 seconds.
`box = driver.find_element(By.ID, "box0")`	Not there yet, so the driver keeps checking until the box appears.
`print("found", box.get_attribute("class"), "after about", round(seconds), "s")`	Returns as soon as it is found, not after the full 2 seconds.

Lesson: Implicit waits

Explicit wait: wait for exactly the condition you need

`revealed = driver.find_element(By.ID, "revealed")`	The input is in the page already, just hidden.
`driver.find_element(By.ID, "reveal").click()`	The page shows it 1 second later.
`wait = WebDriverWait(driver, timeout=2)`	Wait at most 2 seconds. Python checks every 0.5 s.
`wait.until(lambda d: revealed.is_displayed())`	A lambda: keep calling it until it returns something true.
`revealed.send_keys("Displayed")`	Safe now: the input is displayed, so it can take keys.

Lesson: Explicit waits and expected conditions

Ready made conditions, and what a timeout looks like

`from selenium.webdriver.support import expected_conditions as EC`	The usual short name for the expected conditions module.

`box = wait.until(EC.presence_of_element_located((By.ID, "box0")))`	A ready made condition. Note the locator is one tuple. It returns the element.
`field = wait.until(EC.visibility_of_element_located((By.ID, "revealed")))`	Present is not enough for typing. Wait until it is visible.
`WebDriverWait(driver, 1).until(EC.presence_of_element_located((By.ID, "box1")), "the second box never came")`	Nobody clicked again, so box1 never appears. The second argument is our message.

Lesson: Explicit waits and expected conditions

A customised wait: poll faster and retry on an error

`errors = [ElementNotInteractableException]`	The errors that mean not yet, try again.
`wait = WebDriverWait(driver, timeout=2, poll_frequency=0.3, ignored_exceptions=errors)`	Give up after 2 s, check every 0.3 s.
`wait.until(lambda d: revealed.send_keys("Displayed") or True)`	Do the action inside the condition. send_keys returns None, so "or True" reports success.

Lesson: Customising waits, and never mixing them

Proof: mixing implicit and explicit waits stretches the timeout

`driver.implicitly_wait(2)`	Implicit wait: each find may block for 2 seconds.
`WebDriverWait(driver, 3).until(EC.presence_of_element_located((By.ID, "box9")))`	Explicit wait of 3 seconds for an element that never comes.
`print(f"gave up after {time.monotonic() - start:.1f} s, not 3 s")`	Each check inside the explicit wait blocked 2 seconds on the implicit wait.

Lesson: Customising waits, and never mixing them

TABLES

Table: strength · weakness

Strength	Weakness
One line covers every find in the session	It only waits for presence. A present element can still be hidden, covered or disabled
Finds return as soon as the element appears	Every check that something is absent now takes the full wait
Easy to show in a first script	It cannot wait for text, a title, a count or anything that is not a find

Lesson: Implicit waits

Table: default · python webdriverwait

Default	Python WebDriverWait
Polling interval	0.5 s
Errors ignored while looping	NoSuchElementException
Timeout	you must give one
Error at the end	TimeoutException

Lesson: Explicit waits and expected conditions

Table: wait for · python

Wait for	Python
Element exists	`presence_of_element_located`
Element visible	`visibility_of_element_located`
Element clickable	`element_to_be_clickable`
Text present	`text_to_be_present_in_element`
Title contains	`title_contains`
Element gone stale	`staleness_of`

Lesson: Explicit waits and expected conditions

4.2 Windows, tabs, frames and alerts

Switching windows and tabs, working inside iframes, and handling alerts, confirms and prompts.

KEY IDEAS

- **Windows** Each window or tab has a handle. Save the original, switch to the new one, and switch back after close(). newWindow("tab") opens and switches in one step.
- **Frames** Switch into an iframe by element, name or id, or index before finding inside it. defaultContent goes to the top, parentFrame up one level.
- **Title in frames** Get Title always reports the top level page, even from inside a frame.
- **Alerts** alert, confirm and prompt: getText, accept, dismiss, sendKeys. Wait with alert is present when it appears late.
- **Forgotten pop ups** unhandledPromptBehavior defaults to dismiss and notify: the pop up is closed and the next command raises unexpected alert open.

CODE

Follow a link into a new window, come back, open a new tab

`original = driver.current_window_handle`	The id of the current window. It stays the same for the whole session.
`driver.find_element(By.NAME, "windowTwo").click()`	This link has target="_blank", so it opens a new window.
`WebDriverWait(driver, 2).until(EC.number_of_windows_to_be(2))`	Wait until the second window exists.
`for handle in driver.window_handles:`	Every open window, in the order they were opened.
`driver.switch_to.window(handle)`	Switch to the one that is not the original.
`driver.close()`	Close only this window.
`driver.switch_to.window(original)`	Switch back, or the next command fails with no such window.
`driver.switch_to.new_window("tab")`	Selenium 4: open a new tab and switch to it in one step.
`driver.quit()`	Closes every window and tab at once.

Lesson: Windows and tabs

Switch into an iframe three ways and back out

`print("email boxes seen from the top:", len(driver.find_elements(By.ID, "email")))`	The form lives inside the iframe, so the top document sees none.
`driver.switch_to.frame(frame)`	1. Switch with a WebElement: the most flexible way.
`driver.switch_to.default_content()`	Leave the frame: back to the top document.
`driver.switch_to.frame("iframe1-name")`	2. Switch with the frame name or id.
`driver.switch_to.frame(0)`	3. Switch with an index: the first frame of the page.

`print("title inside the frame:", driver.title)`	Get Title always reports the top level page, even from inside a frame.
`print("heading:", driver.find_element(By.ID, "iframe_page_heading").text)`	Outside elements are reachable again.

Lesson: Frames and iframes

Alert, confirm, prompt, a slow alert and one left open

`driver.find_element(By.ID, "alert").click()`	The page calls alert('cheese').
`alert = wait.until(EC.alert_is_present())`	Wait for the alert, then get a handle on it.
`alert.accept()`	Press OK.
`driver.find_element(By.ID, "confirm").click()`	A confirm box: OK or Cancel.
`confirm = driver.switch_to.alert`	Switch to it without waiting, because it opens at once.
`confirm.dismiss()`	Press Cancel.
`driver.find_element(By.ID, "prompt").click()`	A prompt asks for text.
`prompt.send_keys("Selenium")`	Type the answer into the prompt.
`print("page shows:", driver.find_element(By.ID, "text").text)`	The page wrote our answer into a paragraph.
`driver.find_element(By.ID, "slow-alert").click()`	This one opens 200 ms later.
`print("slow:", wait.until(EC.alert_is_present()).text)`	switch_to.alert now would fail, so wait.
`driver.find_element(By.ID, "alert").click()`	Open one more alert and forget about it.
`driver.title`	Any command while an alert is open.
`print("left open:", e.msg.split("\n")[0])`	The default behaviour dismisses it and reports an error.

Lesson: Alerts, confirms and prompts

TABLES

Table: what you want · python

What you want	Python
Current handle	`current_window_handle`
All handles	`window_handles`
Switch	`switch_to.window(h)`
New tab or window	`switch_to.new_window("tab")`
Close current	`close()`

Lesson: Windows and tabs

Table: switch by · example · when to use

Switch by	Example	When to use
WebElement	`switchTo().frame(element)`	Most flexible. Find the frame with any locator.
Name or id	`switchTo().frame("iframe1-name")`	The frame has a stable name or id. If several match, the first one is used. Java and Python accept the string. JavaScript needs the element.
Index	`switchTo().frame(0)`	Position in window.frames. Breaks if frames are added.
Leave all frames	`switchTo().defaultContent()`	Back to the top document.
Up one level	`switchTo().parentFrame()`	For frames inside frames.

Lesson: Frames and iframes

Table: pop up · buttons · what you can do

Pop up	Buttons	What you can do
alert	OK	Read the text, accept
confirm	OK and Cancel	Read the text, accept or dismiss
prompt	OK and Cancel, plus a text box	Read the text, send keys, accept or dismiss. Sending keys replaces the placeholder text

Lesson: Alerts, confirms and prompts

Table: unhandledpromptbehavior · what happens to a forgotten pop up

unhandledPromptBehavior	What happens to a forgotten pop up
dismiss and notify (default)	Dismissed, and the command raises unexpected alert open
accept and notify	Accepted, and the command raises unexpected alert open
dismiss	Dismissed silently
accept	Accepted silently
ignore	Left open, and the command raises unexpected alert open

Lesson: Alerts, confirms and prompts

TIPS AND TRAPS

COMMON MISTAKE After close() you are still pointing at the closed window. The documentation warns that the next command raises no such window. Always switch back to a handle you saved before.

Lesson: Windows and tabs

REMEMBER Recent drivers dismiss beforeunload prompts, the "Leave site?" box, by default. Chapter 2.1 showed where unhandledPromptBehavior is set.

Lesson: Alerts, confirms and prompts

4.3 The Actions API: mouse, keyboard and wheel

Mouse, keyboard and wheel actions, and how a chain becomes ticks.

KEY IDEAS

- **Build then perform** An action chain is sent in one request when perform() is called. Nothing happens before that.
- **Ticks** Each input source has one action per tick. A tick lasts as long as its slowest action.
- **Mouse** doubleClick, contextClick, moveToElement, clickAndHold, dragAndDrop, moveByOffset. The driver remembers the pointer position between chains.
- **Keyboard** keyDown and keyUp for modifiers, sendKeys for text. Use Command on a Mac and Control elsewhere. Release every key you press.
- **Wheel** scrollToElement, scrollByAmount, scrollFromOrigin. Wait for the scroll result, because it can land after perform() returns.

CODE

Double click, right click, hover, drag and drop, and move to a point

`ActionChains(driver).double_click(box).perform()`	Build a chain, then perform() sends it to the browser in one request.
`ActionChains(driver).context_click(box).perform()`	A right click.
`ActionChains(driver).move_to_element(driver.find_element(By.ID, "hover")).perform()`	Move to the centre of the element. Nothing is clicked.
`ActionChains(driver).drag_and_drop(source, target).perform()`	Move to source, press, move to target, release.

`builder = ActionBuilder(driver)`	The lower level builder. Python has no move_to_location on ActionChains.
`builder.pointer_action.move_to_location(8, 12)`	A point measured from the top left of the viewport.
`ActionChains(driver).move_by_offset(13, 15).perform()`	From wherever the pointer is now. The driver remembered it between chains.

Lesson: How actions work, and the mouse

Hold Shift, select all with the right modifier, and type into an element

`.perform()`	Shift is held for abc only. send_keys goes to the box that has focus.
`cmd_ctrl = Keys.COMMAND if sys.platform == "darwin" else Keys.CONTROL`	Select all is Command A on a Mac and Control A elsewhere.
`ActionChains(driver).send_keys_to_element(box, "Selenium!").perform()`	Clicks the element first, then types.

Lesson: Keyboard actions

Scroll to an element, by an amount, and inside a frame

`frame = driver.find_element(By.TAG_NAME, "iframe")`	The page starts with a 5000 px tall block, so the iframe is far below.
`ActionChains(driver).scroll_to_element(frame).perform()`	Scroll until the bottom of the element is in view.
`driver.get("https://www.selenium.dev/selenium/web/scrolling_tests/frame_with_nested_scrolling_frame_out_of_view.html")`	Load it again. A refresh would keep the scroll position.
`ActionChains(driver).scroll_by_amount(0, 300).perform()`	Like turning the wheel: 300 px down from the top left of the viewport.
`WebDriverWait(driver, 2).until(lambda d: d.execute_script("return window.scrollY") ⩾ 300)`	The scroll can land after perform returns, so wait for it.
`ActionChains(driver).scroll_from_origin(ScrollOrigin.from_element(inner), 0, 200).perform()`	Turn the wheel with the pointer over the frame, so the frame scrolls.

Lesson: Scroll wheel actions

TABLES

Table: mouse action · python

Mouse action	Python
Double click	`double_click(el)`
Right click	`context_click(el)`
Hover	`move_to_element(el)`
Press and hold	`click_and_hold(el)`
Drag and drop	`drag_and_drop(a, b)`
Point in the viewport	`ActionBuilder` then `move_to_location(x, y)`
From the current point	`move_by_offset(x, y)`
Release everything	`ActionBuilder(driver).clear_actions()`

Lesson: How actions work, and the mouse

Table: key method · what it does

Key method	What it does
keyDown(key)	Press a key and keep it down. Meant for modifier keys.
keyUp(key)	Release it. Forget this and the key stays down for the next chain.
sendKeys(text)	Type into whatever has focus, one key down and key up per character.
sendKeys(element, text)	Click the element first, then type. Java and Python only.

Lesson: Keyboard actions

Table: wheel action · python

Wheel action	Python
Into view	`scroll_to_element(el)`
By an amount	`scroll_by_amount(0, 300)`
From an element	`scroll_from_origin(ScrollOrigin.from_element(el), 0, 200)`

Lesson: Scroll wheel actions

TIPS AND TRAPS

REMEMBER A pointer move has a duration. In the bindings used here, Python uses 250 ms by default and JavaScript 100 ms. Some drag libraries ignore a move that takes 0 ms, so if a drag does nothing, try a chain with a longer move.

Lesson: How actions work, and the mouse

COMMON MISTAKE Actions type into the focused element. If nothing has focus, the keys go to the page body and the test seems to type into nothing. Element sendKeys focuses the element itself, which is why it is the better choice for plain typing.

Lesson: Keyboard actions

4.4 JavaScript execution and cookies

Running JavaScript in the page, async scripts, and working with cookies.

KEY IDEAS

- **executeScript** The script runs as a function body. Use return to get a value and arguments[n] to read what you pass in. Elements go in and come back as WebElements.
- **Return types** Whole numbers are Long in Java and int in Python, objects are Map and dict, undefined is null or None.
- **Async scripts** executeAsyncScript adds a callback as the last argument. A returned promise is also awaited. The script timeout, 30 s by default, limits both.
- **Cookies** Visit the domain first, then add. httpOnly hides a cookie from page scripts. deleteAllCookies gives a clean start.

CODE

Run JavaScript, pass an element in, and see what type comes back

`print(script, "→", type(driver.execute_script(script)).__name__)`	Run the script as the body of a function and convert the result.
`print("arguments:", driver.execute_script("return arguments[0].innerText + ' / ' + arguments[1]", dinner, 7))`	Extra arguments arrive in the script as arguments[0], arguments[1] and so on.
`driver.execute_script("arguments[0].click();", driver.find_element(By.ID, "updatediv"))`	A click done by JavaScript. It skips the checks a real click makes.

Lesson: Run JavaScript in the page

Asynchronous scripts, promises, the script timeout and a JavaScript error

`waited = driver.execute_async_script("const done = arguments[arguments.length - 1]; setTimeout(() ⇒ done('waited'), 300);")`	Selenium adds a callback as the last argument. The command returns when the script calls it.
`print("promise:", driver.execute_script("return new Promise(r ⇒ setTimeout(() ⇒ r('from promise'), 200));"))`	A returned promise is awaited by the browser before it answers.
`driver.set_script_timeout(0.2)`	How long a script may run, in seconds. The default is 30.
`print("too slow:", e.msg.split("\n")[0])`	The callback came after 500 ms, the limit was 200 ms.
`print("broken script:", e.msg.split("\n")[0])`	A JavaScript error in the page becomes javascript error.

Lesson: Asynchronous scripts and the script timeout

Add, read, protect and delete cookies

`driver.add_cookie({"name": "key", "value": "value"})`	The browser is still on an empty start page with no domain.
`driver.get("https://www.selenium.dev/selenium/web/blank.html")`	Visit the domain first. A small page is enough.
`c = driver.get_cookie("key")`	Read one cookie by name. It comes back as a dict.
`driver.add_cookie({"name": "foo", "value": "bar", "sameSite": "Strict"})`	Any other field goes in the same dict.
`driver.add_cookie({"name": "secret", "value": "s", "httpOnly": True})`	httpOnly hides the cookie from page JavaScript.
`print("missing cookie:", driver.get_cookie("foo"))`	Asking for a cookie that is not there gives None, not an error.
`driver.delete_all_cookies()`	A clean start, for example before logging in as another user.

Lesson: Cookies

TABLES

Table: error · python

Error	Python
The script took longer than the script timeout	TimeoutException
The script threw an error	JavascriptException

Lesson: Asynchronous scripts and the script timeout

Table: field · meaning

Field	Meaning
name, value	Required. The data.
domain, path	Where the cookie is sent. They must match the current page.
secure	Only sent over https.
httpOnly	Hidden from page JavaScript.
sameSite	Strict, Lax or None. Whether the cookie goes with requests started by other sites.
expiry	Seconds since 1970. With no expiry the cookie lasts for the session.

Lesson: Cookies

TIPS AND TRAPS

REMEMBER The script timeout is 30 seconds by default, as chapter 2.1 showed. It covers both sync and async scripts. A short timeout makes a broken async script fail fast.

Lesson: Asynchronous scripts and the script timeout

STEP 5 Build a test suite

5.1 Test frameworks: JUnit 5, pytest and Mocha

Running Selenium inside JUnit 5, pytest and Mocha, with data-driven tests and screenshots on failure.

KEY IDEAS

- **Framework jobs** Selenium drives the browser. JUnit 5, pytest or Mocha find the tests, run setup and teardown, assert and report.
- **Fresh browser** Start the driver in the before each step and quit it in the after each step, so tests stay independent and can run in any order.
- **Teardown always runs** @AfterEach, code after yield and afterEach run also when the test failed, so quit belongs there.
- **Data-driven** @ParameterizedTest with @CsvSource, @pytest.mark.parametrize, or a loop around it() in Mocha. One result per row.
- **Screenshot on failure** Take it after the test failed and before quit: a pytest makereport hook plus fixture.

CODE

A first test class: a fresh browser for every test

`@pytest.fixture`	A fixture is pytest's setup and teardown in one function.
`yield driver`	Everything before yield is setup. The test runs here.
`driver.quit()`	Everything after yield is teardown, and runs also when the test failed.
`def test_page_has_the_right_title(driver):`	Asking for a parameter called driver makes pytest run the fixture.
`assert driver.title == "Web form"`	A plain assert. pytest shows both sides when it fails.

Lesson: Selenium inside a test framework

One test, three rows of data

`@pytest.mark.parametrize("user, password, expected", [`	Each tuple is one run of the test.
`def test_login(driver, user, password, expected):`	pytest passes the columns in as parameters, next to the fixture.
`alert = WebDriverWait(driver, 2).until(EC.alert_is_present())`	The page answers with an alert.

Lesson: Data-driven tests

A failing test that saves a screenshot before the browser closes

`@pytest.hookimpl(wrapper=True, tryfirst=True)`	A hook that sees every test report as pytest builds it.
`setattr(item, "rep_" + report.when, report)`	Keep the report on the test item: rep_setup, rep_call and rep_teardown.
`if request.node.rep_call.failed:`	Did the test body fail?
`name = f"failure-{request.node.name}.png"`	Name the file after the test.
`driver.save_screenshot(name)`	Take the picture before quit.
`def test_title_is_wrong_on_purpose(driver):`	The driver fixture comes from conftest.py.
`assert driver.title == "Web form!"`	Wrong on purpose, so the test fails.

Lesson: A screenshot when a test fails

TABLES

Table: job · pytest

Job	pytest
Before every test	fixture code before `yield`
After every test	fixture code after `yield`
Once for the class or file	`scope="module"` or `"session"`
Check a value	`assert actual == expected`
Run it	`pytest`

Lesson: Selenium inside a test framework

Table: framework · where the screenshot code goes · how it knows the test failed

Framework	Where the screenshot code goes	How it knows the test failed
pytest	The fixture, after `yield` and before `quit()`	A `pytest_runtest_makereport` hook in conftest.py stores the report on the test

Lesson: A screenshot when a test fails

TIPS AND TRAPS

REMEMBER — The Selenium documentation recommends a fresh browser for every test. Then no cookie, login or open dialog can leak from one test into the next, and the tests can run in any order or in parallel. It costs a browser start per test, a price that is almost always worth paying.

Lesson: Selenium inside a test framework

REMEMBER — Keep the data close to the test while it is small. When it grows, move it to a CSV or JSON file: JUnit has @CsvFileSource, and pytest and Mocha can read any file into the same list.

Lesson: Data-driven tests

5.2 Page objects and page components

Page objects, page component objects and the rules for writing them.

KEY IDEAS

- **Page object** A class that is the interface to one page. Tests call its methods, so a change to the page is fixed in one place.
- **No assertions inside** Assertions belong to the test. The only check in a page object is that the right page is loaded.
- **Return the next page** A method that moves to another page returns that page object. A method that stays may return itself.
- **Components** A page component object models a repeated part, such as a field or a product card, and searches only inside its root element.
- **Waits inside** Put waits in the page object, for example a constructor that waits for its title after a navigation.

CODE

Two page objects, one component and three tests

`FIELDS = (By.CSS_SELECTOR, "label.form-label")`	Locators live here and nowhere else.
`if driver.title ≠ "Web form":`	The one check a page object should make: am I on the right page?

`return [Field(e) for e in self.driver.find_elements(*self.FIELDS)]`	Wrap every label element in a Field component.
`class Field:`	A page component: one label and the control inside it.
`return self.root.text.split("\n")[0].strip()`	The first line of the label is its name.
`return self.root.find_element(By.CSS_SELECTOR, "input, textarea, select")`	Search inside this component only.
`return self`	Returning itself allows field(...).fill(...).fill(...).
`WebDriverWait(driver, 3).until(EC.title_is("Web form - target page"))`	The page arrives after a navigation, so wait for it instead of checking once.
`form = WebFormPage.open(driver)`	The test talks to pages, not to locators.
`received = form.submit()`	submit() moves to another page, so it returns that page object.
`assert received.message() == "Received!"`	The assertion stays in the test.
`assert len(fields) == 11`	Eleven labelled controls on this page.
`with pytest.raises(ValueError):`	The constructor checks the page and fails loudly.

Lesson: Write page objects and components

TABLES

Table: without page objects · with page objects

Without page objects	With page objects
Locators are copied into every test that uses the page	Each locator is written once, inside the page object
A test reads as a list of clicks and CSS selectors	A test reads as what the user does: open the form, fill a field, submit
A UI change breaks many tests in many files	A UI change is fixed in one class
Waiting and page checks are repeated	The page object waits for its page once, in its constructor

Lesson: Why page objects

Table: rule from the documentation · where it shows in the example

Rule from the documentation	Where it shows in the example
The public methods represent the services the page offers	`field`, `fields`, `submit`, `message`
Try not to expose the internals of the page	Locators are private. `Field.control()` is private in Java
Generally do not make assertions	Every `assert` is in the test file
The one allowed check: the right page is loaded	The constructors check the title, or wait for it after a navigation
Methods return other page objects, components or themselves	`submit()` returns ReceivedPage, `fill()` returns the Field
A page object need not represent an entire page	`Field` represents one labelled control

Lesson: Write page objects and components

Table: choice · what the documentation or practice suggests

Choice	What the documentation or practice suggests
Same action, different result	Write two methods, for example loginAsValidUser and loginExpectingError, each returning the page that really follows
A part that appears on many pages	A component object such as a header, a menu or a product card, used inside page objects
Where waits go	Inside the page object, so tests do not repeat them
Base class	Shared code such as the driver field and a wait helper. Keep it small

Lesson: Design choices and the limits of page objects

TIPS AND TRAPS

REMEMBER

ReceivedPage waits for its title, while WebFormPage checks the title once. The web form is opened with get(), which returns after the page loads. The received page appears after a click, and chapter 3.3 showed that the next command can run before the new page is there.

Lesson: Write page objects and components

5.3 Good practices and things to avoid

Test design, encouraged practices and the things the documentation says to avoid.

KEY IDEAS

- **Need a browser?** Ask first whether a lighter test can do the job. Browser tests are the most expensive kind.
- **Short and focused** Set up the data, perform a few actions, evaluate the result. Many small independent tests beat one grand journey.
- **Clean start** A new driver starts with a new profile. Use a fresh browser per test and never share test data.
- **Prepare state outside the UI** Log in and create data through APIs, then set cookies, instead of clicking through forms.
- **Avoid** CAPTCHAs, download progress, status codes, third party logins, test order, performance numbers, spidering and 2FA are jobs for other tools or test hooks.

CODE

A new driver starts with a clean profile

`first.add_cookie({"name": "cart", "value": "3 items"})`	State a first test leaves behind: a cookie
`first.execute_script("localStorage.setItem('theme', 'dark')")`	and a value in local storage.
`second = webdriver.Chrome()`	A new driver gets a new, empty profile by default.

Lesson: Encouraged practices

Prepare the state without clicking through the user interface

`session = "abc123"`	In a real suite this value comes from a login API call, not from the login form.
`driver.get("https://www.selenium.dev/selenium/web/blank.html")`	A small page on the domain, only to be allowed to set the cookie.
`driver.get("https://www.selenium.dev/selenium/web/cookies.html")`	Now open the page under test. It reads its cookies on load.

Lesson: Encouraged practices

TABLES

Table: testing type · question it answers · selenium fits?

Testing type	Question it answers	Selenium fits?
Acceptance	Are we building the right product?	Yes, by simulating what the user expects
Functional	Are we building the product right?	Yes
Integration	Do the modules work together, for example order plus payment?	Yes, for flows through the UI
System	Does the whole product work end to end, in an environment like production?	Yes, a few journeys
Performance (load and stress)	How fast is it, and does it survive a peak?	Not as the measuring tool. The documentation points to tools such as JMeter
Regression	Did a change break what worked?	Yes, by running the existing tests again

Lesson: The shape of a good browser test

Table: practice · what the documentation says

Practice	What the documentation says
Avoid sharing state	Do not share test data, clean up stale data, and create a new WebDriver per test
Test independency	Write each test as its own unit, not relying on another test to complete
Mock external services	Removing dependencies on external services improves speed and stability
Improved reporting	Selenium does not report. Use the reports of your test framework, such as xUnit XML or HTML
Locators	Unique ids first, then a well written CSS selector. XPath is flexible but harder to debug and often slower
Page objects and a fluent API	Chapter 5.2. Methods can return the page for chained calls
Domain specific language	Business level methods on top of page objects

Lesson: Encouraged practices

Table: avoid · why · do instead

Avoid	Why	Do instead
CAPTCHAs	They are built to stop automation	Disable them in the test environment, or add a hook that lets tests bypass them
File downloads	The API does not expose download progress	Find the link with Selenium and fetch it with an HTTP library such as curl, using any cookies it needs
HTTP status codes	WebDriver acts like a user, and a status code is rarely the useful detail of a failure	Check what the user sees, such as an error page. Chapter 6.2 shows BiDi network events if you really need codes
Logging in to Gmail or Facebook	Against their terms, slow and unreliable	Use the APIs or test account tools they provide
Test dependency	Tests must run in any order	Make each test set up its own state
Performance testing	Browser start, servers and third parties add noise to every number	Use performance tools, and keep Selenium for function
Link spidering	A browser takes seconds per page	Use curl or a library such as BeautifulSoup
Two factor authentication	Automating it is hard and less secure	A test only token, or 2FA turned off for test users, test IPs or the test environment

Lesson: Things to avoid

TIPS AND TRAPS

REMEMBER Every row has the same shape: Selenium is a tool for what a user does in a browser. When the job is not that, a lighter tool is faster and more reliable.

Lesson: Things to avoid

5.4 Troubleshooting: errors, stale elements and logs

Common errors, stale elements and logging in Java, Python and JavaScript.

KEY IDEAS

- **Error names** Each W3C error has a class per language. Read the name, then the message, then your line in the trace.
- **Stale element** The node behind a reference was replaced, for example by a refresh. Find the element again.
- **Context** An element found in one frame or window is unknown in another. Switch back before using it.
- **Client log** Python: logger selenium at DEBUG.
- **Driver log** ChromeDriver writes COMMAND and RESPONSE lines when its service is given a log file and a verbose level.

CODE

Make an element reference fail three ways, and fix each one

`box = driver.find_element(By.NAME, "my-text")`	The driver gives back a reference to this exact node.
`driver.refresh()`	The page is rebuilt. The old node is gone, a new one with the same name takes its place.
`box = driver.find_element(By.NAME, "my-text")`	The fix: find it again.
`email = driver.find_element(By.ID, "email")`	Found inside the frame.
`print("2. outside its frame:", type(e).__name__)`	The reference is fine, but this context does not know it.
`driver.switch_to.frame("iframe1")`	The fix: switch back to the context where it was found.
`driver.get("https://www.selenium.dev/selenium/web/blank.html")`	Leave the page completely.
`print("3. after leaving the page:", type(e).__name__)`	Go back to the page and find it again.

Lesson: Stale elements and the element store

Turn on the client log and see every command

`class Requests(logging.Handler):`	A tiny handler that prints one line per request.
`m = re.match(r"(GET|POST|DELETE) http://[^/]+(\S*)", record.getMessage())`	Request lines start with the method and the URL.
`print(m.group(1), re.sub("/session/[0-9a-f]{32}", "/session/{id}", m.group(2)))`	Hide the session id, which changes every run.
`log = logging.getLogger("selenium")`	The parent of every Selenium logger in Python.
`log.setLevel(logging.DEBUG)`	Python logs at WARNING by default. Requests are written at DEBUG.

Lesson: Logs: what the client sent and what the driver did

Ask ChromeDriver for its own log

`service = Service(log_output="chromedriver.log", service_args=["--log-level=DEBUG"])`	Where ChromeDriver writes its log, and how much.
`print(line.split("] COMMAND ")[1].split(" ")[0])`	Print the name of each command the driver received.

Lesson: Logs: what the client sent and what the driver did

TABLES

Table: w3c error · python · usual cause and fix

W3C error	Python	Usual cause and fix
no such element	NoSuchElementException	Wrong place, wrong time or a changed locator. Check the page, add an explicit wait, update the locator.
stale element reference	StaleElementReferenceException	The node was replaced, for example by a refresh or a re render. Find it again.
element click intercepted	ElementClickInterceptedException	Something covers the element: a sticky bar, a modal, an animation. Wait for it to go, or scroll.
element not interactable	ElementNotInteractableException	Hidden, zero size or not ready. Wait until visible and enabled.
invalid selector	InvalidSelectorException	The CSS or XPath is broken, or an XPath was passed to a CSS locator.
timeout	TimeoutException	An explicit wait ran out. The message says which condition.
script timeout	TimeoutException	A script took longer than the script timeout.
unexpected alert open	UnexpectedAlertPresentException	An alert was left open. Handle it where it opens.
no such window	NoSuchWindowException	The window was closed and the test did not switch back.
invalid session id	InvalidSessionIdException	The session was quit or crashed. Look for a quit earlier in the test.
session not created	SessionNotCreatedException	Driver and browser versions do not match, or the browser could not start.

Lesson: Read an error the right way

Table: what happened · error in this run · fix

What happened	Error in this run	Fix
Refresh, re render, list re sorted	stale element reference	Find the element again just before you use it
Switched to another frame or window	no such element	Switch back to the context where it was found
Navigated to another page	no such element	Go back to the page and find it again

Lesson: Stale elements and the element store

Table: language · logger · default level · level that shows requests

Language	Logger	Default level	Level that shows requests
Python	logging, under the name selenium	WARNING	DEBUG

Lesson: Logs: what the client sent and what the driver did

TIPS AND TRAPS

REMEMBER Search the error name together with the first line of the message. Ignore the long list of numbers under Stacktrace in Chrome messages. Those are addresses inside ChromeDriver, not your code.

Lesson: Read an error the right way

REMEMBER Page objects that find elements inside their methods, as in chapter 5.2, rarely meet stale elements, because they look the element up fresh each time.

Lesson: Stale elements and the element store

COMMON MISTAKE The documentation shows the JavaScript logger as getLogger("webdriver"). In selenium-webdriver 4.51 that name printed nothing in our run, while driver.http and driver.http.Executor printed every request. In pytest, the documentation adds that logs of passing tests are hidden unless you run pytest -s -p no:logging.

Lesson: Logs: what the client sent and what the driver did

STEP 6 Scale out with Grid

6.1 Selenium Grid 4

Selenium Grid 4 components, roles, remote sessions and parallel runs.

KEY IDEAS

- **Why a Grid** Parallel runs on many browsers, versions and systems, and shorter suites: tests × average time ÷ nodes.
- **Components** Router (4444), New Session Queue, Distributor, Session Map, Node (5555) and Event Bus (4442, 4443).
- **Roles** Standalone for one machine, Hub and Node for most Grids, Distributed for very large ones.
- **Remote driver** Only the driver creation changes. se:name adds metadata, se:cdp gives DevTools through the Grid, and a local file detector sends upload files to the node.
- **Queue and quit** New sessions wait in the queue for a free slot. Always quit, or the slot stays busy until the session timeout.

CODE

Ask the Grid if it is ready and what it offers

`with urllib.request.urlopen("http://localhost:4444/status") as res:`	The same endpoint a driver has, now answered by the Grid.
`print("slot:", slot["stereotype"]["browserName"])`	Each slot can hold one session of this browser.

Lesson: When and why to use a Grid

Three tests, two slots: watch the queue

`driver = webdriver.Remote("http://localhost:4444", options=webdriver.ChromeOptions())`	Returns when the Grid has given this test a slot.
`time.sleep(3)`	Pretend the test takes 3 seconds.
`driver.quit()`	Frees the slot for the next request in the queue.
`with ThreadPoolExecutor(max_workers=3) as pool:`	Three threads, so three tests start at the same moment.
`print("got a browser after:", [f"{t} s" for t in times])`	Two at once, the third only after a slot was free.

Lesson: Components, roles and ports

Run a test on the Grid with a name and a file upload

`options = webdriver.ChromeOptions()`	Options say which browser you want. The Grid picks a node that has it.
`options.set_capability("se:name", "upload on the grid")`	se: capabilities are Grid metadata. se:name shows in the Grid UI.
`driver = webdriver.Remote("http://localhost:4444", options=options)`	The only change from a local test: a URL instead of a local driver.
`print(k + ":", v if k ≠ "se:cdp" else v.replace(driver.session_id, "{id}"))`	se:cdp is a DevTools address that goes through the Grid.
`driver.file_detector = LocalFileDetector()`	Sends the local file to the node, which may be another machine.
`print("attached:", driver.execute_script("return arguments[0].files[0].name", upload))`	The browser on the node has the file.

Lesson: Remote sessions, metadata and files

NUMBERS TO REMEMBER

Suite time on a Grid Exam favourite

$$\text{total time} = \frac{\text{number of tests} \times \text{average test time}}{\text{number of nodes}}$$

From the documentation: 15 tests of 45 s take 11 min 15 s on one node, 2 min 15 s on 5 nodes and 45 s on 15 nodes.

Lesson: When and why to use a Grid

TABLES

Table: component · job · default port

Component	Job	Default port
Router	Entry point for every request. New sessions go to the queue. Commands for existing sessions go straight to the right node, found through the Session Map	4444
New Session Queue	Holds new session requests in first in, first out order, with a timeout	5559
Distributor	Knows every node and its slots. Takes requests from the queue, finds a matching free slot and creates the session	5553
Session Map	Remembers which node runs which session id	5556
Node	Runs the browsers. By default one slot per CPU for Chromium browsers and Firefox, one for Safari	5555
Event Bus	Messages between the other components, cheaper than HTTP. Started first in a distributed Grid	4442, 4443 and 5557

Lesson: Components, roles and ports

Table: role · command · use it for

Role	Command	Use it for
Standalone	`java -jar selenium-server-<version>.jar standalone`	All components in one process on one machine. Local debugging, quick suites, CI
Hub and Node	`... hub` and `... node --hub http://<hub-ip>:4444`	The most used role. Combine machines with different systems and browsers behind one address
Distributed	`... event-bus`, `sessionqueue`, `sessions`, `distributor`, `router`, `node`	Large Grids with more than 100 nodes, each component scaled on its own

Lesson: Components, roles and ports

Table: language · remote driver · file detector

Language	Remote driver	File detector
Python	`webdriver.Remote(gridUrl, options=options)`	`driver.file_detector = LocalFileDetector()`

Lesson: Remote sessions, metadata and files

TIPS AND TRAPS

REMEMBER

A node registers by sending a message on the Event Bus, and the Distributor then calls it over HTTP to confirm. When hub and node are on different machines, the Event Bus ports 4442 and 4443 and the node port must be reachable.

Lesson: Components, roles and ports

6.2 WebDriver BiDi, CDP and Grid features

WebDriver BiDi events, Chrome DevTools Protocol commands and Grid GraphQL.

KEY IDEAS

- **BiDi** A W3C protocol with a WebSocket, so the browser can push events. Turn it on with webSocketUrl, then subscribe to console and page errors.
- **Events, not polling** Handlers are called when events arrive. Wait for them, because they come after the action that caused them.
- **CDP** Chrome and Edge only, no stability promise, the three newest Chrome versions. Use it for Chromium features with no other API.
- **GraphQL** POST a query to /graphql on the Grid for nodes, slots, sessions and the queue size.
- **Grid endpoints** /status, DELETE /session, and drain or remove a node through the Distributor endpoints.

CODE

Hear console messages and page errors as they happen

`options.enable_bidi = True`	Ask for a BiDi WebSocket next to the normal HTTP connection.
`driver.script.add_console_message_handler(lambda e: heard.append(f"console {e.level}: {e.text}"))`	Called for every console.log, console.error and so on.
`driver.script.add_javascript_error_handler(lambda e: heard.append(f"page error: {e.text}"))`	Called for every uncaught error in the page.
`WebDriverWait(driver, 5).until(lambda _: len(heard) == 3)`	Events are pushed, not returned, so wait until all three arrived.

Lesson: WebDriver BiDi: the browser talks back

Send Chrome DevTools commands: read the version and change the time zone

`driver = webdriver.Chrome()`	execute_cdp_cmd exists on Chrome and Edge drivers only.
`version = driver.execute_cdp_cmd("Browser.getVersion", {})`	Any CDP method by name, with a dict of parameters.
`driver.execute_cdp_cmd("Emulation.setTimezoneOverride", {"timezoneId": "Asia/Kolkata"})`	Something WebDriver Classic has no command for.

Lesson: The Chrome DevTools Protocol

Ask the Grid exactly what you need with GraphQL

`req = urllib.request.Request("http://localhost:4444/graphql", data=json.dumps({"query": query}).encode(), method="POST")`	GraphQL over HTTP: one POST with the query as a string.
`q = "{ grid { nodeCount, totalSlots, maxSession, sessionCount, sessionQueueSize } }"`	Ask only for these fields.
`print("busy:", ask(q)["grid"])`	One session is running now.
`print("names:", [json.loads(s["capabilities"]).get("se:name") for s in sessions])`	The se:name metadata is in the session capabilities.

Lesson: Grid features: GraphQL and endpoints

TABLES

Table: how to switch it on · python

How to switch it on	Python
Capability	`options.enable_bidi = True`
Console and page errors	`driver.script.add_console_message_handler`

Lesson: WebDriver BiDi: the browser talks back

Table: webdriver classic · cdp · webdriver bidi

	WebDriver Classic	CDP	WebDriver BiDi
Standard	W3C Recommendation	Chrome only, no stability promise	W3C, in progress
Browsers	All	Chrome and Edge	Chrome, Edge and Firefox, growing
Style	Request and response over HTTP	Commands and events over WebSocket	Commands and events over WebSocket
Use it for	Every normal test step	Chromium features with no other API	Events, logs, network, and the future of Selenium

Lesson: The Chrome DevTools Protocol

Table: endpoint · what it does

Endpoint	What it does
GET /status	Current state of the Grid and every node, with slots and sessions
POST /graphql	Ask for exactly the fields you need: grid, nodesInfo, sessionsInfo, session(id)
DELETE /session/<id>	End a session and free its slot
POST /se/grid/distributor/node/<node-id>/drain	Stop a node after its running sessions end, with no new sessions. Needs the registration secret header
DELETE /se/grid/distributor/node/<node-id>	Remove a node from the Distributor. Its running sessions continue

Lesson: Grid features: GraphQL and endpoints

TIPS AND TRAPS

COMMON MISTAKE BiDi is still growing. The documentation lists log, network, script and browsing context features, and some network APIs are marked as not yet stable in some bindings, for example the Python docs example uses an underscore method for intercepts. Check the documentation page for your binding before you build on a feature.

Lesson: WebDriver BiDi: the browser talks back

REMEMBER The documentation calls out two popular CDP examples with limited value: emulated geolocation, because most sites use the IP address, and overriding device metrics, because Chrome mobile emulation in the options is the better tool.

Lesson: The Chrome DevTools Protocol

STEP 7 Revise and practise

7.1 Revision: predict, debug and design

Predict the output, find the bug, design and sizing questions, a cheat sheet in three languages and a mock quiz.

KEY IDEAS

- **Predict, then run** The driver does what the protocol says, not what the code seems to say. Offsets add, Shift changes case, a refresh makes references stale.

- **Cause before fix** Name why a test is slow or flaky before changing it. Sleeps, mixed waits and missing quit calls are the usual causes.
- **Layers** Tests, page objects and components, a driver factory and support code. Each change should touch one layer.
- **Size with the formula** Suite time = tests × average time ÷ nodes, with about one CPU and 1 GB of RAM per browser.
- **Know your binding** Time is a Duration in Java, seconds in Python and milliseconds in JavaScript, and a missing cookie gives null, None or an error.

NUMBERS TO REMEMBER

Explicit wait poll Exam favourite

500 ms

The default check interval in Java and Python.

Lesson: Cheat sheet in three languages

Page load timeout

300,000 ms

Five minutes by default.

Lesson: Cheat sheet in three languages

Script timeout

30,000 ms

Thirty seconds by default.

Lesson: Cheat sheet in three languages

TABLES

Table: question shape · what it looks like · what earns the marks · lesson

Question shape	What it looks like	What earns the marks	Lesson
Predict the output	A short program and the question: what does it print, or which error comes back	Knowing what the driver really does, not what the code seems to say	2
Find the bug	A test that is slow, flaky or fails, and the question: why, and how do you fix it	Naming the cause first, then a fix that removes it	3
Design and sizing	Plan a framework, size a Grid, triage a flaky suite	Clear layers, numbers from a formula, a reason for each choice	4
Write it in another language	The same step in Java, Python or JavaScript	The right method name and the right unit for time	5

Lesson: How Selenium skills are tested

Table: step · python

Step	Python
Start Chrome	`webdriver.Chrome()`
Open a page	`driver.get(url)`
Find one element	`driver.find_element(By.ID, "q")`
Type	`el.send_keys("text")`
Wait for an element	`WebDriverWait(driver, 5).until(EC.visibility_of_element_located((By.ID, "q")))`
Into a frame	`driver.switch_to.frame(el)`
Back to the page	`driver.switch_to.default_content()`
Run a script	`driver.execute_script("return 1")`
Add a cookie	`driver.add_cookie({{"name": "k", "value": "v"}})`
Use a Grid	`webdriver.Remote(command_executor="http://localhost:4444", options=webdriver.ChromeOptions())`
End the session	`driver.quit()`

Lesson: Cheat sheet in three languages

TIPS AND TRAPS

REMEMBER

Use the Revise lessons in order the first time, then use the Quiz and Worked problems tabs as a timed mock test. Each answer says which chapter to reread if you got it wrong.

Lesson: How Selenium skills are tested

COMMON MISTAKE

A good bug answer has three parts: what the code really does, why that breaks, and a fix that removes the cause. A retry that hides the failure is not a fix.

Lesson: Find the bug